Day 42: Weekly Revision

Day 42 — Weekly Revision

1. Day Number

Day 42

2. Topic Name

Revision of Days 36–41

Today you will revise:

- functions returning `None`
- `remove()`
- `pop()`
- `sort()`
- writing to a text file
- reading from a text file

---

3. Connection

During the last six days, you learned several small Python features.

You learned how to:

```
Validate a value
      ↓
Return value or None
      ↓
Work with lists
      ↓
Remove items
      ↓
Sort values
      ↓
Save data to a file
      ↓
Read data back
```

Today we combine these ideas into one very small transaction exercise.

---

4. Revision Summary of Days 36–41

Day 36 — Function Returning None

You learned that sometimes a function cannot return a valid result.

For example:

```
def check_age(age):
    if age > 0:
        return age
    return None
```

Here:

```
check_age(20)
```

returns:

```
20
```

but:

```
check_age(-5)
```

returns:

```
None
```

Main idea

`None` means:

> There is currently no valid value/result.

It is different from:

```
0
""
False
```

because those are actual values.

Day 37 — remove()

`remove()` removes a specific value from a list.

Example:

```
fruits = ["apple", "banana", "orange"]
fruits.remove("banana")
```

Now:

```
["apple", "orange"]
```

Remember:

```
remove(value)
```

means:

> Find this value and remove it.

You should normally check whether the value exists first:

```
if "banana" in fruits:
    fruits.remove("banana")
```

Day 38 — pop()

`pop()` normally removes the **last item** from a list.

Example:

```
numbers = [10, 20, 30]
last_number = numbers.pop()
```

After this:

```
numbers
```

contains:

```
[10, 20]
```

and:

```
last_number
```

contains:

```
30
```

So `pop()` does two things:

```
removes item
+
returns removed item
```

Day 39 — sort()

`sort()` arranges items inside the existing list.

Example:

```
prices = [300, 100, 200]
prices.sort()
```

Result:

```
[100, 200, 300]
```

By default, numbers are sorted from:

```
small → large
```

Day 40 — File Writing

You learned how to save information into a file.

Basic structure:

```
with open("example.txt", "w") as file:
    file.write("Hello\n")
```

Important:

```
"w"
```

means **write mode**.

And:

```
\n
```

means start a new line.

Day 41 — File Reading

You learned how to read information from a file.

Basic structure:

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Important:

```
"r"
```

means **read mode**.

`strip()` removes unwanted whitespace such as the newline at the end of each line.

---

5. Important Topics

For today's revision, focus on these six ideas:

```
None
remove()
pop()
sort()
file write
file read
```

Think of their purposes like this:

Topic	Purpose
`None`	represent no valid result
`remove()`	remove a particular value
`pop()`	remove and return an item
`sort()`	arrange list values
`"w"`	write data to file
`"r"`	read data from file

---

6. Foundational Notes

A. Lists are mutable

A list can be changed after it is created.

Example:

```
items = ["A", "B"]
```

You can add:

```
items.append("C")
```

You can remove:

```
items.pop()
```

You can sort:

```
items.sort()
```

This is why lists are useful for things such as:

- carts
- products
- transaction history
- prices
- tasks

B. remove() and pop() are different

Suppose:

```
transactions = ["Deposit", "Purchase", "Refund"]
```

With:

```
transactions.remove("Purchase")
```

you are saying:

> Remove `"Purchase"`.

But with:

```
transactions.pop()
```

you are saying:

> Remove the last item.

So:

```
remove() → know the value
pop()    → usually care about position
```

C. Never blindly pop() an empty list

This is dangerous:

```
transactions = []
transactions.pop()
```

because there is nothing to remove.

Instead, think:

```
Is the list empty?

Yes → don't pop
No  → pop
```

A simple check is:

```
if transactions:
```

An empty list behaves like `False`.

D. sort() modifies the list

Consider:

```
amounts = [500, 100, 300]
amounts.sort()
```

The same `amounts` list becomes:

```
[100, 300, 500]
```

You usually do not need:

```
amounts = amounts.sort()
```

because `sort()` itself returns `None`.

That is an important beginner point.

E. None should be checked clearly

Suppose a function may return `None`.

You can check:

```
if result is None:
```

This clearly communicates:

> I am checking whether there is no valid result.

F. with manages the file

Instead of manually opening and closing files, Python commonly uses:

```
with open(...) as file:
```

The general idea is:

```
Open file
   ↓
Work with file
   ↓
Leave with-block
   ↓
Python closes file
```

G. Writing replaces file contents in "w" mode

When you use:

```
open("transactions.txt", "w")
```

Python prepares the file for writing.

If the file already contains information, `"w"` mode normally replaces the existing contents.

For now, remember:

```
"w" → write fresh content
"r" → read existing content
```

H. Each transaction should have a newline

If you write:

```
file.write(transaction)
```

several times, the data could become:

```
DepositPurchaseRefund
```

Instead, each transaction should end with:

```
"\n"
```

giving:

```
Deposit
Purchase
Refund
```

---

7. Easy Example

This is only a small example of the individual concepts, not today's complete solution.

Suppose we have some scores:

```
scores = [30, 10, 20]
```

Sort them:

```
scores.sort()
```

Now:

```
[10, 20, 30]
```

Suppose we have some tasks:

```
tasks = ["email", "meeting", "report"]
```

Remove the last one:

```
last_task = tasks.pop()
```

Now:

```
tasks = ["email", "meeting"]
last_task = "report"
```

And you could save simple text using:

```
with open("tasks.txt", "w") as file:
    # write items here
```

Notice how several small concepts can work together.

---

8. Revision Problem Statement

Create a small transaction program.

Your program should:

1. Create an empty transaction list.
2. Add three simple transaction strings.
3. Remove the last transaction using `pop()`.
4. Create a separate list containing simple transaction amounts.
5. Sort the amounts from low to high.
6. Write the final transaction list to a text file.
7. Keep the program very simple.

For example, conceptually you might have:

```
Transactions:
Deposit 500
Purchase 200
Refund 100
```

Then remove the last transaction.

Final transaction list:

```
Deposit 500
Purchase 200
```

Separately, you might have amounts such as:

```
[500, 200, 100]
```

which should become:

```
[100, 200, 500]
```

Then save the **final transactions** to a text file.

---

9. Concepts Used

You will use:

Variables

To store values.

```
transactions
amounts
```

Lists

To keep multiple transactions and amounts.

append()

To add transactions.

Conceptually:

```
transactions.append(...)
```

pop()

To remove the last transaction.

Conditional statement

To make sure the list is not empty before using `pop()`.

sort()

To sort amount numbers.

Function

To organize parts of the program.

For example, conceptually:

```
add transactions
remove last transaction
sort amounts
write transactions
```

None

Useful when handling invalid amounts.

File writing

Using:

```
open()
```

with:

```
"w"
```

with

To safely work with the file.

Loop

To write each transaction one at a time.

---

10. Thought Process

Before writing Python code, break the problem into tiny steps.

Step 1: What data do I need?

You need a transaction list.

Start mentally with:

```
transactions = empty list
```

Step 2: How many transactions?

The problem asks for three.

So:

```
empty list
    ↓
add transaction 1
    ↓
add transaction 2
    ↓
add transaction 3
```

Result:

```
[transaction1, transaction2, transaction3]
```

Step 3: Should I call pop() immediately?

First ask:

```
Is transaction list empty?
```

If no:

```
remove last transaction
```

If yes:

```
do nothing / handle empty list
```

Step 4: What happens after pop()?

Before:

```
Transaction 1
Transaction 2
Transaction 3
```

After:

```
Transaction 1
Transaction 2
```

The third transaction has been removed.

Step 5: What about amounts?

Create a separate numeric list.

For example:

```
500
100
300
```

Before sorting:

```
[500, 100, 300]
```

After sorting:

```
[100, 300, 500]
```

Step 6: What should be written to the file?

The problem says:

> Write final transactions.

So don't write the original three-item transaction list.

Write the list **after** `pop()`.

Step 7: How should transactions appear in the file?

Prefer:

```
Transaction 1
Transaction 2
```

rather than:

```
Transaction 1Transaction 2
```

Therefore each transaction needs a newline.

---

11. Pseudocode

Pseudocode is not Python. It describes the logic.

```
START

CREATE empty transaction list

ADD first transaction
ADD second transaction
ADD third transaction

IF transaction list is not empty
    REMOVE last transaction using pop

CREATE list of amount numbers

SORT amount numbers from low to high

DISPLAY sorted amounts

OPEN transaction file in write mode

FOR each transaction in final transaction list
    WRITE transaction
    WRITE newline

CLOSE file automatically

END
```

If you want to include the Day 36 `None` concept as well:

```
CREATE function to validate amount

IF amount is positive
    RETURN amount
ELSE
    RETURN None
```

Then only use valid amounts.

---

12. Suggested Solving Approach — Functional Approach

Instead of putting everything together immediately, think in terms of small functions.

A simple design could be:

```
main
 |
 +--> add_transactions()
 |
 +--> remove_last_transaction()
```

```
|
+--> sort_amounts()
|
+--> write_transactions()
```

You do **not** need many complicated functions.

A possible conceptual structure is:

```
def remove_last_transaction(...):
    ...

def sort_amounts(...):
    ...

def write_transactions(...):
    ...

def main():
    ...
```

Why use functions?

Because each function has one small responsibility.

For example:

```
remove_last_transaction()
```

should focus only on removing the last transaction.

It should not also:

- calculate discounts
- print receipts
- create products
- perform unrelated work

This makes your program easier to understand.

---

13. Easy Edge Cases

Edge Case 1 — Empty List

Suppose:

```
transactions = []
```

You should not blindly use:

```
transactions.pop()
```

Think:

```
Is list empty?
    ↓
Yes
    ↓
Don't pop
```

Edge Case 2 — Invalid Amount

Suppose an amount is:

```
-100
```

If negative amounts are considered invalid for this exercise, your validation function could conceptually return:

```
None
```

Flow:

```
amount
  ↓
positive?
 /      \
yes      no
 |        |
amount   None
```

Then check the result before using it.

Edge Case 3 — Amount 0

Depending on your rule, `0` may also be invalid.

If the rule is:

```
amount must be positive
```

then:

```
0 → None
```

Edge Case 4 — Empty Amount List

Suppose:

```
amounts = []
```

Sorting it is safe.

Conceptually:

```
[] → sort → []
```

There is simply nothing to rearrange.

Edge Case 5 — Already Sorted Amounts

Suppose:

```
[100, 200, 300]
```

After sorting:

```
[100, 200, 300]
```

No problem.

Edge Case 6 — Duplicate Amounts

Suppose:

```
[300, 100, 300]
```

Sorted:

```
[100, 300, 300]
```

`sort()` does not remove duplicates.

Edge Case 7 — Empty File

If no transactions are written, the file may simply be empty.

Reading an empty file with:

```
for line in file
```

means the loop runs zero times.

That's normal.

---

14. Common Mistakes to Avoid

Mistake 1: Calling pop() without checking the list

Avoid:

```
transactions.pop()
```

when you do not know whether the list contains anything.

First think:

```
if transactions:
```

Mistake 2: Confusing remove() and pop()

Remember:

```
remove("Tea")
```

means:

> Remove the value `"Tea"`.

While:

```
pop()
```

means:

> Remove the last item.

Mistake 3: Assigning the result of sort()

This is a common beginner mistake:

```
amounts = amounts.sort()
```

After this, `amounts` may become `None`.

Why?

Because:

```
list.sort()
```

changes the existing list and returns `None`.

Think:

```
amounts.sort()
```

instead.

Mistake 4: Forgetting \n while writing

Without newline:

```
Deposit 500Purchase 200
```

With newline:

```
Deposit 500
Purchase 200
```

Mistake 5: Using "r" when trying to write

Remember:

```
"r" → read
"w" → write
```

Mistake 6: Using "w" when you only wanted to read

`"w"` can replace existing contents.

So pay attention to the mode.

Mistake 7: Forgetting strip() when reading lines

A file line often contains an invisible newline:

```
"Deposit 500\n"
```

Using:

```
line.strip()
```

can give:

```
"Deposit 500"
```

Mistake 8: Comparing incorrectly with None

Prefer:

```
if result is None:
```

rather than treating `None` like an ordinary number or string.

Mistake 9: Writing the removed transaction too

Remember today's sequence:

```
Add 3
  ↓
pop last
  ↓
2 remain
  ↓
write final list
```

The removed item should not appear in the final file.

---

15. Quick Self-Check Questions

Question 1

What is the difference between:

```
items.remove("Book")
```

and:

```
items.pop()
```

Question 2

What happens to:

```
numbers = [30, 10, 20]
numbers.sort()
```

What should `numbers` contain afterward?

Question 3

Why is this dangerous?

```
transactions = []
transactions.pop()
```

What should you check first?

Question 4

What is the purpose of:

```
None
```

in a function?

For example:

```
invalid amount → ?
```

Question 5

What is the difference between:

```
open("data.txt", "w")
```

and:

```
open("data.txt", "r")
```

16. Hint Only

Build the program in this order:

```
transactions = []
       ↓
append 3 transactions
       ↓
check whether list has items
       ↓
pop()
       ↓
create amount list
       ↓
sort()
       ↓
open file using "w"
       ↓
loop through final transactions
       ↓
write transaction + "\n"
```

For the functional approach, think about small functions such as:

```
validate_amount(amount)
remove_last_transaction(transactions)
sort_amounts(amounts)
write_transactions(transactions)
```

And for Day 36's concept:

```
if amount is valid
    return amount
otherwise
    return None
```

Your main challenge today: combine the concepts you already know without adding new complexity. Do not worry about user menus, databases, classes, or advanced error handling yet.